Auto-Vectorization of Audio DSP Kernels: Advanced Code Layouts, Mathematical Reformulations, and Compiler Optimization Mechanics

Theoretical Foundations of Loop-Level Vectorization vs Superword-Level Parallelism

Automatic vectorization within modern compiler middle-ends, specifically Clang/LLVM and GCC, relies on two primary instruction-widening passes: Loop-Level Vectorization (LLV) and Superword-Level Parallelism (SLP).[1] While both passes are designed to generate Single Instruction Multiple Data (SIMD) instructions, they target distinct code structures and undergo entirely different intermediate representation (IR) analysis pipelines.[1]

Loop-Level Vectorization (LLV)

LLV is designed to find and exploit data-level parallelism across successive iterations of a loop.[1] The LLV pass transforms loops by widening scalar instructions to execute over a Vector Factor (VF) of elements simultaneously.[6] Mathematically, this transformation maps the loop index i from a unit-stride recurrence $i \leftarrow i + 1$ to a vector-stride recurrence $i \leftarrow i + VF$.[6]
The compiler uses polyhedral loop modeling to analyze the iteration space and verify that no loop-carried data dependencies exist.[4] In this model, loop iterations are represented as points within a multi-dimensional integer lattice.[4] A loop is vectorizable if the execution schedule can be mapped to parallel scheduling vectors without violating the partial ordering enforced by the data dependency vectors.[4]

Superword-Level Parallelism (SLP)

SLP, by contrast, operates on straight-line scalar code within a single basic block.[4] Instead of parallelizing across loop iterations, the SLP pass merges independent, isomorphic scalar instructions into a single vector instruction.[4] Isomorphic instructions are defined as those that perform the exact same operation (e.g., addition, multiplication) in the same order on distinct data elements.[4]
The SLP vectorizer works bottom-up by scanning the basic block's IR to locate "vectorization seeds".[5] These seeds are typically consecutive memory store operations or reduction tree roots.[5] Once a seed is identified, the algorithm builds a directed acyclic graph (DAG) by recursively traversing the use-def chains upward.[5] If the compiler determines that the instructions in the DAG are independent and isomorphic, it replaces the scalar instructions with

equivalent vector operations.[5]

Lane-Level Parallelism (LLP)

Recent compiler developments introduce Lane-Level Parallelism (LLP), which generalizes SLP by allowing the vectorization of non-isomorphic operations on adjacent lanes of a vector register.[9] LLP is particularly valuable for complex arithmetic common in audio (e.g., Fast Fourier Transforms or complex filter stages).[9] LLP allows instructions to execute multiple distinct operations (such as alternating addition and subtraction) on different vector lanes.[9]
This is achieved by mapping the scalar AST to specialized hardware operations, such as the addsub instruction family, and utilizing vector shuffle blends to route inputs to their correct target lanes.[9]

Commutative Reordering and LSLP

A significant limitation of standard SLP is its fragility when encountering commutative operations with mismatched operand layouts.[5] If the scalar instructions in a basic block do not present their inputs in a strictly matched order, the bottom-up SLP graph building phase fails, and the compiler falls back to scalar code generation.[5]
To address this, advanced optimization frameworks utilize Look-ahead SLP (LSLP).[5] LSLP executes a short-depth look-ahead analysis of the operand DAG.[5] By evaluating whether a commutative reordering of operands upstream will prevent a memory load alignment

mismatch or register shuffle downstream, LSLP restructures the AST before vector emission, allowing the compiler to generate clean SIMD load-and-pack sequences.[5]

Memory Topology: Layout-Guided Code Generation (SoA vs AoS)

The performance of auto-vectorized code is heavily dependent on the memory layout of the input and output data streams.[13] Audio data structures are typically organized into one of two top-level topologies: Array of Structures (AoS) or Structure of Arrays (SoA).[13]

Structure of Arrays (SoA) vs Array of Structures (AoS)

In an AoS layout, multi-channel audio samples are interleaved sequentially:

$$\text{AoS} =$$

For multi-channel streams, AoS organizes the individual channels as a single structure, grouping the fields of a single sample contiguously in memory.[13] While this provides excellent cache locality for sequential operations on a single multi-channel sample, it presents a major barrier to the compiler's auto-vectorizer.[13]

If a processing loop applies an operation to only a single channel (e.g., scaling the left channel), the compiler must access memory with a stride of 2.[13] Because SIMD load and store operations require contiguous memory addresses to achieve single-cycle throughput, strided memory accesses force the compiler to generate complex data unpacking sequences or fall back to scalar execution.[15]

In a SoA layout, each channel's buffer is segregated into independent, parallel arrays:

$$\text{SoA}_L = [L_0, L_1, \ldots, L_{N-1}], \quad \text{SoA}_R =$$

SoA maps directly to the parallel execution lanes of SIMD hardware.[13] Sequential loop iterations read contiguous memory locations ($i, i+1, i+2$), allowing the compiler to emit direct, aligned or unaligned vector moves (vmovdqu or vmovdqa) with zero layout reorganization or register shuffling overhead.[14]

Memory Layout Feature	Array of Structures (AoS)	Structure of Arrays (SoA)
Memory Access Stride	Non-unit stride (equal to channel count) [13]	Unit stride (stride-1, contiguous) [13]

SIMD Load Instruction	Unpacking loads / Gather instructions [15]	Direct aligned/unaligned vector moves [18]
Vector Layout Overhead	High (demands shuffles like punpcklqdq) [15]	Zero (data is pre-aligned with lanes) [14]
Cache Line Utilization	Low if processing a single channel [13]	Optimal; 100% of fetched cache lines are used [14]
Compiler Cost Evaluation	Often rejected due to packing/unpacking cost [15]	Highly profitable; easily vectorized [2]

Advanced Compiler Analysis Mechanics

To optimize code for the auto-vectorizer, developers must understand the exact AST-level and IR-level analyses executed by the compiler frontend and middle-end.

Scalar Evolution (SCEV)

LLVM's SCEV pass models how the values of variables change inside loops using non-recursive, symbolic mathematical expressions.[23] SCEV represents loop induction variables and pointer arithmetic as Add Recurrence (addrec) expressions, formatted as [23]:

$$\{\text{BasePointer}, +, \text{Step}\}_{<\text{Loop}>}$$

If the step size of an addrec matches the byte width of the element type, the memory access is categorized as a stride-1, unit-stride access, which is highly profitable for vectorization.[7]
If the loop bounds or pointer increments are non-constant, the compiler cannot verify stride-1 access at compile-time.[1] It must then generate complex address calculation code, perform loop peeling, or insert runtime bounds checks, which often causes the cost model to reject the vectorization pass.[1]

Pointer Aliasing and Alias Analysis (AA)

The C++ standard permits distinct pointers of the same type to refer to overlapping memory locations.[1] Under this rule, writing to an output array could modify the data in an input array during the same loop iteration.[1]
If the compiler cannot prove that pointers are disjoint, it generates a runtime check in the loop prelude.[1] This check compares the physical addresses of the pointers:

$$|\text{PtrA} - \text{PtrB}| < VF \times \text{sizeof}(\text{Type})$$

If the pointers overlap, the program branches to a slow scalar version of the loop.[1] Decorating pointers with the __restrict__ keyword or __restrict qualifier informs the Alias Analysis pass (such as BasicAliasAnalysis or ScopedNoAliasAA) that the pointers are completely disjoint.[1] This metadata removes the memory-dependency edges from the compiler's dependence graph.[4] The loop scheduler is then free to reorder memory operations, optimize register allocation, and strip out the runtime check and its associated scalar fallback loop.[1]

Alignment Analysis

SIMD load and store instructions achieve maximum throughput when the target memory addresses are aligned to the vector register boundaries (e.g., 32-byte alignment for AVX2, 64-byte for AVX-512).[26] If pointer alignment is unknown at compile-time, the compiler generates unaligned loads (vmovdqu) or inserts a "peeling" loop.[4]

Loop peeling executes a series of initial scalar iterations until the target pointer is aligned, followed by the main vectorized loop.[4] This peeling process increases code size and introduces runtime branching overhead.[1]

Asserting pointer alignment via alignas or compiler alignment intrinsics (such as __builtin_assume_aligned or std::assume_aligned) updates the base pointer's alignment properties within the SCEV representation, allowing the compiler to bypass peeling entirely and generate direct, aligned vector moves (vmovdqa).[4]

Masked Residual Loops and Tail Folding

When a loop's trip count N is not a multiple of the vector factor VF, the remaining $N \pmod{VF}$ elements cannot be processed in a full vector step.[1] Compilers handle these remaining elements by generating a scalar epilogue (or remainder) loop.[1]

However, generating a scalar epilogue loop increases code size and requires conditional branch checks.[1]

Modern vector architectures (such as AVX-512 and ARM SVE) support "tail folding," which avoids scalar epilogues by executing the final iteration using a masked vector loop.[2] The compiler generates a vector mask that disables the inactive lanes at the end of the buffer, preventing out-of-bounds writes while maintaining a single, uninterrupted vector execution path.[2]

Dependency Resolution Tactics for Recursive Loop-Carried Recurrences

A classic challenge in audio DSP auto-vectorization is the 2-pole Infinite Impulse Response (IIR)

filter, such as a standard Biquad filter.[26] The time-domain difference equation for a Direct Form II Biquad is written as [29]:

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]$$

This equation has a loop-carried Read-After-Write (RAW) data dependency: computing the output $y[n]$ requires the immediate past outputs $y[n-1]$ and $y[n-2]$.[17] Because $y[n-1]$ must be fully computed before the next iteration can begin, standard loop-level vectorizers cannot parallelize the loop and must fall back to scalar execution.[17]
To vectorize these loops, the code must be mathematically restructured to expose parallel execution paths.[29]

Multi-Channel Interleaving

In professional audio, the most robust and mathematically exact method to vectorize recursive filters is Multi-Channel Interleaving.[26] Instead of attempting to parallelize a single mono stream, the audio engine is restructured to process C independent audio channels (such as stereo pairs or multi-bus mixers) in parallel.[26]
By organizing the filter coefficients and state variables as a Structure of Arrays (SoA) across channels, the calculations for each channel become completely independent.[14]
If the number of interleaved channels matches the vector width ($C = VF$), the recursive terms for each channel are loaded into distinct lanes of the same vector register.[26] The filter updates can then be executed simultaneously across all lanes using packed vector instructions, resolving the loop-carried dependency bottleneck without modifying the filter's numerical response.[26]

Scattered Look-Ahead Pipelining (SLA)

For single-channel recursive filters where multi-channel interleaving is not viable, developers can use Scattered Look-Ahead (SLA) pipelining.[31] SLA transforms the recursive difference equation to eliminate intermediate feedback dependencies by inserting canceling poles and zeros into the filter's transfer function.[31]
To illustrate, consider a first-order recursive loop:

$$y[n] = a \cdot y[n-1] + x[n]$$

This equation has a single-cycle loop-carried dependency.[17] To create M independent

concurrent computations, the filter's transfer function $H(z) = 1/(1 - az^{-1})$ is multiplied by an augmented canceling polynomial $D(z)$ [31]:

$$D(z) = 1 + az^{-1} + a^2z^{-2} + \cdots + a^{M-1}z^{-(M-1)}$$

The transformed transfer function becomes [31]:

$$H(z) = \frac{1 + az^{-1} + a^2z^{-2} + \cdots + a^{M-1}z^{-(M-1)}}{1 - a^Mz^{-M}}$$

In the time domain, this is expressed as [32]:

$$y[n] = a^My[n - M] + \sum_{k=0}^{M-1} a^kx[n - k]$$

By setting the look-ahead depth M to equal the vector factor ($M = VF$), the recursive dependency is projected back by VF samples.[32] The feedforward summation term:

$$u[n] = \sum_{k=0}^{M-1} a^kx[n - k]$$

is non-recursive and can be completely vectorized using standard LLV.[31] Once $u[n]$ is computed, the recursive feedback equation $y[n] = a^My[n - M] + u[n]$ can be executed in M parallel, independent scalar loops, achieving high-throughput vector execution.[32]

For a 2nd-order Biquad filter, look-ahead pipelining requires solving a system of matrix equations to compute the canceling coefficients.[31] The transformed second-order difference equation is written as [31]:

$$y[n] = q_0 y[n-M] + q_1 y[n-M-1] + \sum_{i=0}^{M-1} h[i]x[n-i]$$

where q_0 and q_1 are the look-ahead feedback coefficients, and $h[i]$ represents the impulse response of the feedforward section.[31] This reformulation projects the feedback dependency out of the immediate vector register space, enabling parallelized execution at the expense of a linear increase in arithmetic operations.[31]

Parallel Prefix (Kogge-Stone) Formulations

Recursive loops can also be reformulated as associative binary operations, allowing them to be solved using parallel prefix networks like the Kogge-Stone adder.[33]

A parallel prefix network processes a recurrence relation of size N using a tree-like structure.[35] This reduces the critical path computational complexity from a serial $\mathcal{O}(N)$ to a parallel $\mathcal{O}(\log_2 N)$.[35]

To apply this to a first-order recursive filter $y[n] = a[n] \cdot y[n-1] + x[n]$, the state transition is modeled as an associative prefix operation.[34] The prefix equations are evaluated across vector registers using parallel shifts and fused multiply-add (FMA) instructions.[34] At each step s of the Kogge-Stone network, the intermediate variables are shifted by 2^s and combined [33]:

$$P_{\text{new}} = P_{\text{current}} \times (P_{\text{current}} \ll 2^s)$$

$$G_{\text{new}} = G_{\text{current}} + P_{\text{current}} \times (G_{\text{current}} \ll 2^s)$$

This approach allows a highly recursive scalar mono loop to be parallelized across vector registers, although it requires additional arithmetic operations and complex register masking.[34]

State-of-the-Art Academic Literature and Compilers Review

A review of academic literature and modern production compilers highlights the techniques

used to automate numerical code vectorization.[31]

Framework / Source	Primary Vectorization Approach	Target Architectures	Key Optimization Technique
Faust Compiler [39]	Stage-decoupling multi-loop generation (--vectorize)	x86 (AVX), ARM (SVE), WebAssembly	Splitting complex feedback loops into simpler, vector-communicating sub-loops.[39]
SLEEF Math Library [38]	Branch-free SIMD transcendental function evaluations	x86, ARM, RISC-V (RVV)	Link-Time Optimization (LTO) and inline headers to eliminate branch overhead.[38]
VEGEN / LLP Vectorizer [9]	Lane-Level Parallelism DAG packing	Cross-platform LLVM IR	Broadening SLP to support non-isomorphic operations on adjacent lanes.[9]
LSLP Algorithm [5]	Graph-based look-ahead operand reordering	SSE, AVX, NEON	Short-depth look-ahead to reorder commutative operations and prevent memory shuffles.[5]
ULA Matrix Approach [31]	Universal Look-Ahead polynomial matrix solvers	Custom DSP / ASIC / FP-SIMD	Solving look-ahead coefficients via matrix inversion and multiplication.[31]

The Faust compiler generated C++ code demonstrates how restructuring high-level code can assist the compiler.[39] Rather than trying to vectorize a complex, single-sample loop with deep feedback paths, Faust's vector code generator (-vec) splits the algorithm into multiple simple loops.[39]

These loops communicate through local vector buffers, often restricted to a block size of 32 or

64 samples.[39] By separating feedforward operations from recursive feedback calculations, the compiler is presented with simple, unit-stride loops that Clang and GCC can easily auto-vectorize.[39]

Research on SLEEF (SIMD Library for Evaluating Elementary Functions) highlights the importance of branch elimination and memory access optimization.[38] Standard mathematical function calls containing scalar assembly or conditional branches prevent compiler loop vectorization.[38]

SLEEF provides vectorized versions of all C99 math functions, maintaining high performance and portability.[38] SLEEF can be integrated with Link-Time Optimization (LTO) or as inlined headers to minimize function call overhead, allowing Clang and GCC to vectorize loops containing transcendental functions without scalar fallbacks.[38]

Comparative performance studies of auto-vectorization in Clang and GCC on RISC-V Vector Extensions (RVV) and ARM SVE highlight the sensitivity of compiler cost models.[21] Tests from the TSVC suite show that Clang and the Intel C++ Compiler (ICX) frequently generate vector scatter/gather instructions for complex 2D array strides.[21]

However, because scatter/gather operations carry high hardware latencies, they often perform worse than unvectorized code or basic loop unrolling.[15] This demonstrates that successful auto-vectorization requires careful code structure modifications to avoid relying on complex compiler-generated memory shuffles.[15]

DSP Code Transformation Blueprint

The following section provides concrete C++ transformation blueprints for three common audio DSP use cases. Each case shows naive, unvectorizable code contrasted with optimized, compiler-friendly refactored code.

Case 1: Parallel Multi-Channel Buffer Mixer

Naive Implementation (Vectorization Inhibited)

This implementation fails to vectorize efficiently due to potential pointer aliasing, unaligned memory access, and non-constant loop bounds that require runtime checks.[1]

```
C++
struct Channel {
    float* data;
};

void mix_buffers_naive(Channel* inputs, int num_channels, float* output, int num_samples)
```

```
  // Naive processing loop with aliasing risks and unaligned access
  for (int c = 0; c < num_channels; ++c) {
    for (int i = 0; i < num_samples; ++i) {
      output[i] += inputs[c].data[i];
    }
  }
}
```

Optimized Implementation (Guaranteed Auto-Vectorization)

This optimized version resolves the compiler frontend's analysis bottlenecks. Pointers are decorated with the __restrict__ qualifier, which eliminates pointer aliasing hazards and removes the need for runtime checks.[1]
The buffers are asserted as aligned to 32-byte boundaries (for AVX2) using __builtin_assume_aligned, bypassing loop peeling.[4] The processing loop operates on a block size that is a constant multiple of the vector factor, allowing the compiler to omit scalar epilogue loops.[1]

```
C++
#include <new>
#include <cstdint>

constexpr int kMixBlockSize = 256;  // Guaranteed multiple of AVX2/AVX-512 register sizes

void mix_buffers_optimized(
  float* __restrict__ const * __restrict__ const input_channels,
  int num_channels,
  float* __restrict__ const output,
  int num_samples)
{
  // Constrain loop bounds to static constants to simplify Scalar Evolution (SCEV)
  const int aligned_samples = (num_samples / kMixBlockSize) * kMixBlockSize;

  for (int c = 0; c < num_channels; ++c) {
    // Assert pointer alignment to eliminate loop peeling
    const float* __restrict__ const in_lane =
        static_cast<const float*>(__builtin_assume_aligned(input_channels[c], 32));
    float* __restrict__ const out_lane =
```

```
        static_cast<float*>(__builtin_assume_aligned(output, 32));

    #pragma clang loop vectorize(enable) interleave(enable)
    #pragma GCC ivdep
    for (int i = 0; i < aligned_samples; ++i) {
        // Emitted as zero-overhead aligned moves (vmovdqa) and packed adds (vaddps)
        out_lane[i] += in_lane[i];
    }
}
```

Case 2: Unrolled Phase-Accumulating Wavetable Oscillator

Naive Implementation (Vectorization Inhibited)

This oscillator contains an branch conditional (if) to wrap the phase accumulator, which breaks vector lane execution efficiency.[28]
Additionally, the float-to-int conversion and the resulting non-contiguous table lookups prevent the compiler from generating unit-stride loads, leading to scalarization.[22]

```
C++
struct NaiveOscillator {
    float phase;
    float frequency_multiplier;
    float* wavetable;
    int table_size;
};

void process_osc_naive(NaiveOscillator* osc, float* output, int num_samples) {
    for (int i = 0; i < num_samples; ++i) {
        int index = (int)osc->phase;
        output[i] = osc->wavetable[index];

        osc->phase += osc->frequency_multiplier;
        if (osc->phase >= osc->table_size) {
            osc->phase -= osc->table_size;
        }
```

Optimized Implementation (Guaranteed Auto-Vectorization)

To vectorize the phase accumulator, we eliminate the branching logic using bitwise masking.[25] By constraining the table size to a power of two, we can replace the conditional wrapping with a bitwise AND on integer phase representations.[29]
We model the phase accumulator as an induction vector using fixed-point arithmetic, which is highly compatible with integer vector units.[1] The lookup is vectorized using explicit compiler-guided linear interpolation operations that require no branches.[22]

```
C++
#include <cstdint>

struct OptimizedOscillator {
    uint32_t phase_fractional; // 32-bit fixed-point phase representation
    uint32_t phase_increment;  // Phase increment per sample
    const float* __restrict__ wavetable; // Guaranteed power-of-two size (e.g., 1024)
};

constexpr uint32_t kOscTableSize = 1024;
constexpr uint32_t kOscTableMask = kOscTableSize - 1;
constexpr uint32_t kOscFractionShift = 22; // 32 - log2(1024) bits

void process_osc_optimized(
    OptimizedOscillator* __restrict__ const osc,
    float* __restrict__ const output,
    int num_samples)
{
    float* __restrict__ const out_lane =
        static_cast<float*>(__builtin_assume_aligned(output, 32));
    const float* __restrict__ const table =
        static_cast<const float*>(__builtin_assume_aligned(osc->wavetable, 32));

    uint32_t local_phase = osc->phase_fractional;
    const uint32_t local_inc = osc->phase_increment;
```

```
#pragma clang loop vectorize(enable) interleave(enable)
#pragma GCC ivdep
for (int i = 0; i < num_samples; ++i) {
    // Branchless phase indexing via bitwise mask
    uint32_t index_0 = (local_phase >> kOscFractionShift) & kOscTableMask;
    uint32_t index_1 = (index_0 + 1) & kOscTableMask;

    // Convert fixed-point fraction to a float scale;
    float y1 = table[index_1];

    // Linear interpolation executed as a branchless fused multiply-add (FMA)
    out_lane[i] = y0 + frac * (y1 - y0);

    local_phase += local_inc;
}

osc->phase_fractional = local_phase;
}
```

Case 3: Recursive Biquad Filter using Channel-Interleaved Block Processing

Naive Implementation (Vectorization Inhibited)

The standard Biquad implementation processes a single channel with feedback states updated recursively.[29] This creates a loop-carried dependency that blocks auto-vectorization, forcing the compiler to emit scalar assembly.[17]

```
C++
struct BiquadCoeffs {
    float b0, b1, b2, a1, a2;
};

struct BiquadState {
    float w1, w2;
};
```

```
void process_biquad_naive(
    const BiquadCoeffs* coeff,
    BiquadState* state,
    const float* input,
    float* output,
    int num_samples)
{
    for (int i = 0; i < num_samples; ++i) {
        // Recursive Direct Form II formulation with strict RAW dependency
        float w0 = input[i] - coeff->a1 * state->w1 - coeff->a2 * state->w2;
        output[i] = coeff->b0 * w0 + coeff->b1 * state->w1 + coeff->b2 * state->w2;

        state->w2 = state->w1;
        state->w1 = w0;
    }
}
```

Optimized Implementation (Guaranteed Auto-Vectorization)

To resolve this feedback bottleneck, we refactor the filter structure to process 8 independent audio channels in parallel (corresponding to a 256-bit AVX2 vector register containing 8 floats).[26]
The coefficients and filter states are laid out as a Structure of Arrays (SoA), meaning that the states of Channel 0 to Channel 7 are grouped contiguously.[13] This restructuring eliminates the loop-carried dependency within the loop iteration space, allowing the compiler's loop vectorizer to generate packed instructions.[26]

```
C++
#include <new>

struct alignas(32) BiquadCoeffsSoA {
    float b0;
    float b1;
    float b2;
    float a1;
    float a2;
};
```

```
struct alignas(32) BiquadStateSoA {
    float w1[8];
    float w2[8];
};

void process_biquad_soa(
    const BiquadCoeffsSoA* __restrict__ const coeff,
    BiquadStateSoA* __restrict__ const state,
    const float* __restrict__ const * __restrict__ const inputs,
    float* __restrict__ const * __restrict__ const outputs,
    int num_samples)
{
    alignas(32) float local_w1[8];
    alignas(32) float local_w2[8];

    // Load states into aligned local variables
    #pragma clang loop vectorize(enable) interleave(enable)
    for (int c = 0; c < 8; ++c) {
        local_w1[c] = state->w1[c];
        local_w2[c] = state->w2[c];
    }

    for (int i = 0; i < num_samples; ++i) {
        alignas(32) float x[8];
        #pragma clang loop vectorize(enable) interleave(disable)
        for (int c = 0; c < 8; ++c) {
            x[c] = inputs[c][i];
        }

        alignas(32) float y[8];

        // This loop processes the 8 channels in parallel.
        // Because there is no cross-channel dependency, it is vectorized.
        #pragma clang loop vectorize(enable) interleave(disable)
        for (int c = 0; c < 8; ++c) {
            float w0 = x[c] - coeff->a1[c] * local_w1[c] - coeff->a2[c] * local_w2[c];
            y[c] = coeff->b0[c] * w0 + coeff->b1[c] * local_w1[c] + coeff->b2[c] * local_w2[c];

            local_w2[c] = local_w1[c];
            local_w1[c] = w0;
        }

        // Store outputs back to channel buffers
```

```
    #pragma clang loop vectorize(enable) interleave(disable)
    for (int c = 0; c < 8; ++c) {
        outputs[c][i] = y[c];
    }
}

// Write back updated states to memory structures
#pragma clang loop vectorize(enable) interleave(enable)
for (int c = 0; c < 8; ++c) {
    state->w1[c] = local_w1[c];
    state->w2[c] = local_w2[c];
}
}
```

Cost Model Profiling and TargetTransformInfo Tuning

To decide whether a loop is "profitable" to vectorize, Clang/LLVM and GCC evaluate the execution costs of the scalar and vectorized variants.[2] In LLVM, the LoopVectorizationCostModel pass queries the TargetTransformInfo (TTI) interface to calculate these costs.[6]

Cost Calculation Mechanics

The compiler assigns a target-specific numerical cost (C) to each intermediate representation (IR) instruction under both scalar and vectorized execution scenarios [5]:

$$C_{\text{scalar\_loop}} = \sum C(\text{Scalar Instructions}) \times \text{Trip Count}$$

$$C_{\text{vector\_loop}} = \frac{\sum C(\text{Vector Instructions}) \times \text{Trip Count}}{VF} + C_{\text{prelude}} + C_{\text{epilog}}$$

C_{prelude} represents the cost of executing runtime checks, copying values to vector registers, and loop peeling.[1] C_{epilogue} is the cost of running a remainder scalar loop to process fractional iterations.[1]

If $C_{\text{vector\_loop}} < C_{\text{scalar\_loop}}$, vectorization is deemed profitable.[5] However, several edge cases can lead to code bloat and performance regressions.[44]

Preventing Bloated Code Generation

To prevent the compiler from emitting bloated loop peeling, alignment checks, and epilogue loops, developers should optimize the following parameters:

Enforced Static Loop Bounds

If the loop bounds are dynamic, the compiler cannot verify that the trip count is a multiple of the vector factor.[1] By constraining loop bounds to constants (e.g., using constexpr block sizes), the compiler can determine that the trip count is a multiple of the vector width.[7]
This allows the compiler to completely omit the scalar epilogue loop, reducing code size and instruction cache pressure.[1]

Vector Width and Interleave Factor Overrides

LLVM uses two critical parameters: the Vectorization Factor (VF) and the Interleave Factor (IF).[1] VF determines the width of the vector registers used, while IF controls the unrolling factor for pipelining operations.[1]
If the cost model fails to select optimal values for these factors, developers can override them using command-line arguments:

```bash
# Force a Vectorization Factor of 8 (e.g., AVX2 float8)
-mllvm -force-vector-width=8

# Force an Interleave Factor of 2 to pipeline independent instructions
-mllvm -force-vector-interleave=2
```

Alternatively, developers can use #pragma clang loop directives to specify these parameters directly in the source code.[22]

Cost Model Overrides in GCC

GCC's tree vectorizer uses four distinct cost models [2]:

- dynamic: The default model, which estimates both compile-time and run-time checks to determine profitability.[2]
- cheap: Vectorizes only if the entire loop can be replaced with a vectorized version without a scalar epilogue loop.[2]
- very cheap: Minimizes code bloat by vectorizing only when no runtime checks or

remainder loops are generated.[2]

- unlimited: Assumes vectorization is always profitable, bypassing the cost heuristic entirely.[2]

In latency-critical audio applications with small block sizes (e.g., 32 or 64 samples), the cost model may determine that vectorization is unprofitable due to prologue/epilogue overhead.[40] To force vectorization in these cases, developers can use the OpenMP #pragma omp simd directive.[2] This directive instructs the compiler to use the unlimited cost model, assuming vectorization is profitable and bypassing internal heuristic checks.[2]

Compiler Diagnostics Diagnostic Matrix

The following table provides a diagnostic guide to compiler optimization warnings and errors encountered when compiling audio DSP loops, paired with the exact refactoring patterns required to resolve them.

Diagnostic Remark / Error Message	Compiler	Architectural Root Cause	Code Refactoring Pattern to Resolve
loop not vectorized: loop-carried dependency prevented vectorization [19]	LLVM (Clang) / GCC	The loop body contains a Read-After-Write (RAW) dependency where iteration i reads data modified by iteration $i -$.[17]	Apply Scattered Look-Ahead pipelining to defer feedback dependencies [31], or restructure the code to process multiple independent audio channels in parallel.[26]
loop not vectorized: cannot identify array bounds [41]	LLVM (Clang)	The compiler cannot prove that the output pointer does not overlap with any input pointers, raising a potential alias hazard.[1]	Decorate all array pointer arguments with the __restrict__ qualifier.[1] If the data is read-only, declare the source as const float* __restrict__ const.[19]
loop not vectorized:	LLVM (Clang) / GCC	The loop contains a	Eliminate branching

control flow is not understood by vectorizer [28]		conditional branch, an if statement, or a switch statement that cannot be flattened.[22]	in the inner loop.[25] Replace conditionals with branchless ternary operators ? : (which compile to vector select instructions) or bitwise masks.[25]
vectorization unprofitable: compiler cost model deemed vectorization unprofitable [43]	LLVM (Clang) / GCC	The scalar loop execution cost is estimated to be lower than the vector loop setup, peeling, and epilogue overhead.[22]	Use #pragma clang loop vectorize(enable) or apply OpenMP #pragma omp simd to bypass cost heuristics and force vectorization.[2] Align buffers to eliminate peeling costs.[4]
not vectorized: complicated access pattern [20]	GCC	The loop accesses memory via non-contiguous offsets, nested pointer-indirection, or non-unit strides.[13]	Flatten multi-dimensional arrays into single contiguous buffers.[20] Convert the layout from Array of Structures (AoS) to Structure of Arrays (SoA) to restore unit strides.[13]
not vectorized: unsupported use in stmt [20]	GCC	The loop uses a variable defined in a previous iteration of the loop that is not recognized as a valid reduction or induction.[1]	Ensure that all intermediate loop variables are locally scoped within the loop block, allowing them to be mapped to vector registers.[1]
loop not vectorized: loop contains a	LLVM (Clang)	The loop contains conversions	Ensure consistent numeric types (all

non-vectorizable conversion operation [25]		between incompatible types (e.g., float to integer) that lack native SIMD instruction support.[25]	float or all double) across the processing path to eliminate mid-loop conversions.[22]

Works cited

1. Auto-Vectorization in LLVM — LLVM 23.0.0git documentation - LLVM.org, accessed May 22, 2026, https://llvm.org/docs/Vectorizers.html
2. Vectorization optimization in GCC - Red Hat Developer, accessed May 22, 2026, https://developers.redhat.com/articles/2023/12/08/vectorization-optimization-gcc
3. Optimizing with auto-vectorization - Arm Developer, accessed May 22, 2026, https://developer.arm.com/documentation/102699/0100/Optimizing-with-auto-vectorization
4. An Automatic Superword Vectorization in LLVM, accessed May 22, 2026, https://people.cs.nycu.edu.tw/~wuuyang/homepage/papers/mypaper-ZAKK.pdf
5. (PDF) Look-ahead SLP: auto-vectorization in the presence of commutative operations, accessed May 22, 2026, https://www.researchgate.net/publication/323507137_Look-ahead_SLP_auto-vectorization_in_the_presence_of_commutative_operations
6. lib/Transforms/Vectorize/LoopVectorize.cpp Source File - LLVM.org, accessed May 22, 2026, https://llvm.org/doxygen/LoopVectorize_8cpp_source.html
7. CS 6120: LLVM Loop Autovectorization - Cornell: Computer Science, accessed May 22, 2026, https://www.cs.cornell.edu/courses/cs6120/2019fa/blog/llvm-autovec/
8. lib/Transforms/Vectorize/LoopVectorize.cpp - platform/external/llvm - Git at Google, accessed May 22, 2026, https://android.googlesource.com/platform/external/llvm/+/2c3e005/lib/Transforms/Vectorize/LoopVectorize.cpp
9. VeGen: A Vectorizer Generator for SIMD and Beyond - Charith Mendis, accessed May 22, 2026, https://charithmendis.com/assets/pdf/21-asplos-vegen.pdf
10. Automatic vectorization - Wikipedia, accessed May 22, 2026, https://en.wikipedia.org/wiki/Automatic_vectorization
11. Retrofitting Control Flow Graphs in LLVM IR for Auto Vectorization - arXiv, accessed May 22, 2026, https://arxiv.org/html/2510.04890v1
12. slpvectorizer::BoUpSLP Class Reference - LLVM, accessed May 22, 2026, https://cs6340.cc.gatech.edu/LLVM8Doxygen/classllvm_1_1slpvectorizer_1_1BoUpSLP.html
13. Structure of Arrays vs Array of Structures - c++ - Stack Overflow, accessed May 22, 2026, https://stackoverflow.com/questions/17924705/structure-of-arrays-vs-array-of-structures

14. Struct of Arrays (SoA) vs Array of Structs (AoS) - Performance - Julia Discourse, accessed May 22, 2026, https://discourse.julialang.org/t/struct-of-arrays-soa-vs-array-of-structs-aos/30015
15. Compiler support for semi-manual AoS-to-SoA conversions with data views - arXiv, accessed May 22, 2026, https://arxiv.org/html/2405.12507v1
16. Structures of Arrays vs Arrays of Structures? - CUDA Programming and Performance - NVIDIA Developer Forums, accessed May 22, 2026, https://forums.developer.nvidia.com/t/structures-of-arrays-vs-arrays-of-structures/13581
17. Using Automatic Vectorization, accessed May 22, 2026, http://www.physics.ntua.gr/~konstant/HetCluster/intel12.1/compiler_f/main_for/optaps/common/optaps_vec_use.htm
18. C++ SIMD Auto-Vectorization: Why Array Access Patterns Matter | NP Blog - Nazarii Piontko, accessed May 22, 2026, https://www.npiontko.pro/2026/01/24/simd-auto-vectorization
19. Clang vectorization - LLVM Discussion Forums, accessed May 22, 2026, https://discourse.llvm.org/t/clang-vectorization/35893
20. What do gcc's auto-vectorization messages mean? - Stack Overflow, accessed May 22, 2026, https://stackoverflow.com/questions/13505524/what-do-gccs-auto-vectorization-messages-mean
21. Comparison of Vectorization Capabilities of Different Compilers for X86 and ARM CPUs, accessed May 22, 2026, https://arxiv.org/html/2502.11906v1
22. Auto-Vectorization in LLVM - ROCm Documentation, accessed May 22, 2026, https://rocm.docs.amd.com/projects/llvm-project/en/latest/LLVM/llvm/html/Vectorizers.html
23. LLVM: Scalar evolution - nikic's Blog, accessed May 22, 2026, https://www.npopov.com/2023/10/03/LLVM-Scalar-evolution.html
24. llvm::TargetTransformInfo Class Reference, accessed May 22, 2026, https://llvm.org/doxygen/classllvm_1_1TargetTransformInfo.html
25. Vectorization of loops (GNAT User's Guide for Native Platforms) - GCC, the GNU Compiler Collection, accessed May 22, 2026, https://gcc.gnu.org/onlinedocs/gnat_ugn/Vectorization-of-loops.html
26. A Biquad Implementation using Advanced Vector Extensions - LIGO DCC, accessed May 22, 2026, https://dcc.ligo.org/public/0179/T2100460/003/T2100460-v3.pdf
27. Vectorization and Parallelization of Loops in C/C++ Code - Jackson State University, accessed May 22, 2026, https://www.jsums.edu/robotics/files/2016/12/FECS17_Proceedings-FEC3555.pdf
28. Efficiently searching an array with GCC, Clang and ICC : r/cpp - Reddit, accessed May 22, 2026, https://www.reddit.com/r/cpp/comments/qbwkjr/efficiently_searching_an_array_with_gcc_clang_and/
29. Vectorizing IIR Filters: What are you Recursing? - Ayan Shafqat, accessed May 22,

2026, https://shafq.at/vectorizing-iir-filters.html
30. Where is the loop-carried dependency here? - c++ - Stack Overflow, accessed May 22, 2026, https://stackoverflow.com/questions/14007805/where-is-the-loop-carried-dependency-here
31. A universal look-ahead algorithm for pipelining IIR filters - SciSpace, accessed May 22, 2026, https://scispace.com/pdf/a-universal-look-ahead-algorithm-for-pipelining-iir-filters-527qf76sbo.pdf
32. Chapter 10: Pipelined and Parallel Recursive and Adaptive Filters Keshab K. Parhi, accessed May 22, 2026, http://people.ece.umn.edu/users/parhi/SLIDES/chap10.pdf
33. Kogge–Stone adder - Wikipedia, accessed May 22, 2026, https://en.wikipedia.org/wiki/Kogge%E2%80%93Stone_adder
34. Parallel Prefix Algorithms - Chessprogramming wiki, accessed May 22, 2026, https://www.chessprogramming.org/Parallel_Prefix_Algorithms
35. DESIGN AND IMPLEMENTATION OF FASTER PARALLEL PREFIX KOGGE STONE ADDER - ijeetc, accessed May 22, 2026, https://www.ijeetc.com/v3/v3n1/13_A0141_(p.116-121).pdf
36. KOGGE-STONE ADDER: A SCALABLE DESIGN SOLUTION TO HIGH-THROUGHPUT DIGITAL SYSTEMS - ICTACT Journals, accessed May 22, 2026, https://ictactjournals.in/paper/IJME_Vol_11_Iss_3_Paper_2_2151_2156.pdf
37. Automatic Vectorization - Cornell Virtual Workshop, accessed May 22, 2026, https://cvw.cac.cornell.edu/vector/intro/auto-vectorization
38. SLEEF Vectorized Math Library, accessed May 22, 2026, https://sleef.org/
39. Using the Compiler - Faust Documentation - ijc, accessed May 22, 2026, https://ijc8.me/faustdoc/manual/compiler/
40. Using the Compiler - Faust Documentation - Grame, accessed May 22, 2026, https://faustdoc.grame.fr/manual/compiler/
41. Loop Vectorization: Diagnostics and Control - The LLVM Project Blog, accessed May 22, 2026, https://blog.llvm.org/2014/11/loop-vectorization-diagnostics-and.html
42. A comparison of auto-vectorization performance between GCC and LLVM for the RISC-V vector extension - kth .diva, accessed May 22, 2026, https://kth.diva-portal.org/smash/get/diva2:1905944/FULLTEXT01.pdf
43. SLP and VPlan - IR & Optimizations - LLVM Discussion Forums, accessed May 22, 2026, https://discourse.llvm.org/t/slp-and-vplan/86663
44. Regression in SLP vectorization cost model · Issue #198029 · llvm/llvm-project · GitHub, accessed May 22, 2026, https://github.com/llvm/llvm-project/issues/198029
45. Loop Vectorize: Testing cost model driven transformations - LLVM Discussion Forums, accessed May 22, 2026, https://discourse.llvm.org/t/loop-vectorize-testing-cost-model-driven-transformations/43251
46. VecTrans: LLM Transformation Framework for Better Auto-vectorization on

High-performance CPU - arXiv, accessed May 22, 2026, https://arxiv.org/html/2503.19449v1
47. Clang Language Extensions — Clang 23.0.0git documentation, accessed May 22, 2026, https://clang.llvm.org/docs/LanguageExtensions.html